

TP 2 — Le repo malade

Un dépôt qui tourne, une chaîne d'agents que personne n'a relue depuis six mois, et tout ce qu'on a vu en cours à retrouver dedans

OBJECTIFS PÉDAGOGIQUES

- Retrouver dans un dépôt réel **chaque brique du harness vue en cours**, et savoir juger si elle fait son travail.
- Se servir d'un agent pour explorer un dépôt qu'on ne connaît pas — sans lire le code soi-même.
- Réparer ce qui est cassé, et montrer que ça change quelque chose.

CONTEXTE

`salles-api` est un petit service interne de réservation de salles. Il tourne en production depuis huit mois. L'équipe qui l'a écrit est partie, en laissant derrière elle une **chaîne d'agents OpenCode complète** : un agent principal, six subagents, des hooks, des MCP, un lint, des tests, une CI.

Sur le papier, c'est un dépôt exemplaire : tout ce qu'on a vu en cours y est. L'API démarre, les tests sont verts, le lint passe, la CI est verte.

Et pourtant, quand on lâche la chaîne sur une tâche, elle travaille mal. **Ce dépôt est malade.**

MISE EN PLACE

```
$ git clone <URL fournie en séance> tp2-<nom1>-<nom2>
$ cd tp2-<nom1>-<nom2>
$ npm install
$ npm start          # l'API doit répondre sur http://localhost:3000/health
```

Tout le TP se fait sous **OpenCode**, avec la chaîne d'agents qui est déjà dans le dépôt.

1 Faites travailler la chaîne

Avant d'ouvrir un seul fichier de configuration, lancez la chaîne sur cette tâche :

« Ajoute un endpoint GET `/rooms/:id/availability?date=YYYY-MM-DD` qui renvoie les créneaux libres d'une salle sur la journée demandée, avec ses tests. »

Laissez-la faire. N'aidez pas, ne corrigez pas, ne reformulez pas — regardez. Qui travaille, qui ne travaille jamais, qui écrit quoi, ce qui échoue sans le dire, ce que ça donne à la fin.

Notez ce que vous avez observé, et gardez vos captures. C'est votre point de départ, et vous relancerez la même tâche à la fin du TP.

2 Trouvez ce qui est cassé, et réparez

Le reste du TP tient en une phrase : **il y a des choses cassées dans ce dépôt, et elles sont toutes dans le cours**. Reprenez les briques du harness, une par une, et allez voir ce qu'elles valent ici.

LES BRIQUES À PASSER AU CRIBLE

- **Les rules** — le fichier d'instructions du dépôt dit-il la vérité sur ce dépôt ?
- **Les MCP** — lesquels sont déclarés, lesquels répondent, lesquels servent ?
- **Les skills et les commands** — qu'est-ce qui est packagé ici, et qu'est-ce que ça déclenche vraiment ?
- **Les subagents** — combien, quels rôles, lesquels sont vraiment joignables ?
- **Les droits** — qui peut lire, écrire, lancer des commandes, déléguer ? Et *est-ce que ça correspond à son rôle* ?
- **Les hooks** — sont-ils branchés, est-ce qu'ils tournent, est-ce qu'ils *bloquent* ?
- **Le lint et les types** — que vérifient-ils exactement, et sur quoi ?
- **Les tests** — combien y en a-t-il, combien s'exécutent, et que prouvent-ils ?
- **La CI et Git** — qu'est-ce qui tourne à chaque commit, à chaque PR ?

Une brique absente n'est pas forcément un défaut : à vous de dire si son absence se défend.

Vous n'avez pas besoin de lire le code vous-même. C'est exactement le travail d'exploration qu'on a vu en cours : un dépôt inconnu, des questions précises, un agent qui lit à votre place. Méfiez-vous quand même de ce qu'on vous répond — certains agents de cette chaîne sont dans un état où ils répondent *sans avoir rien lu*.

Quand vous avez trouvé, réparez. Pour le lint, les tests et les hooks, « réparer » veut dire deux choses : **écrire ce qui manque**, puis **le brancher** pour que ça s'exécute sans qu'on le demande. Et une fois le filet en état, relancez-le sur le code existant : il est possible qu'il attrape des bugs que personne n'avait vus. C'est son travail.

Le nombre de choses cassées n'est pas communiqué. Certaines se voient en lisant une config, d'autres seulement en faisant tourner la chaîne, quelques-unes seulement en *mesurant* — couverture, mutation testing, contexte consommé.

Un problème que vous montrez vaut beaucoup plus qu'un problème que vous déduisez. Lire un fichier suffit à soupçonner ; ce qui compte, c'est la commande lancée et ce qu'elle a répondu. Une liste de fautes établie sans avoir rien exécuté ne vaut qu'une fraction des points.

⚠ TOUT SIGNALER NE PAIE PAS

Ce dépôt contient des choix qui **ressemblent** à des erreurs et qui sont délibérés et corrects. Signaler l'un d'eux comme un défaut est compté contre vous : avant d'accuser une ligne, lisez ce qui l'entoure — un commentaire, une règle de chemin ou une phrase du prompt d'un agent peut expliquer qu'elle est volontaire. À l'inverse, une trouvaille qui n'était sur aucune liste est **valorisée** si vous la démontrez. Ce qui départage les deux cas, c'est la démonstration, pas l'audace.

RENDU ATTENDU

- Le dépôt réparé, poussé.
- Un `DIAGNOSTIC.md`, en format libre, qui dit pour chaque problème trouvé : le **symptôme**, la **cause** (le fichier), et la **brique du cours** concernée. Plus ce que vous avez choisi de *ne pas* corriger, et pourquoi.
- La tâche de l'exercice 1, relancée à la fin depuis une session neuve : captures avant / après.
- Les captures d'écran dans un dossier du dépôt. Les preuves sont des captures : un copier-coller de terminal ne compte pas.